



UNIVERSITÀ DI PARMA

Debuggin with Code::Blocks

*The most
careful tool
placed pr*



*g tool is still
h judiciously*

Brian Kernighan

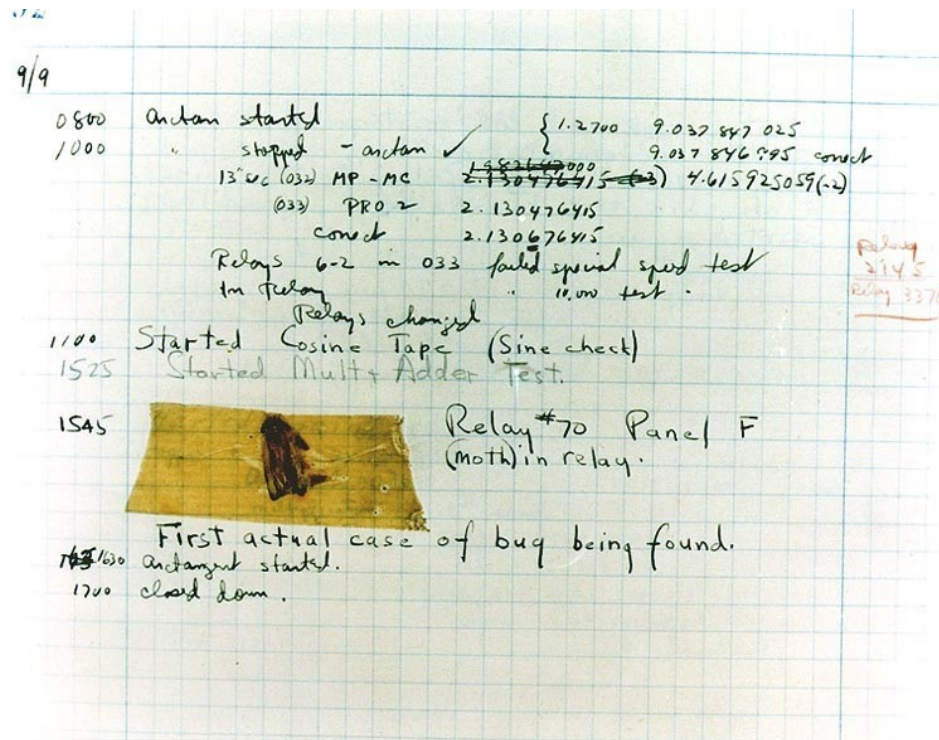
- Background
- Basic C issues
- Tools and Techniques
- Code::Blocks debug tool
- Hints

SUMMARY



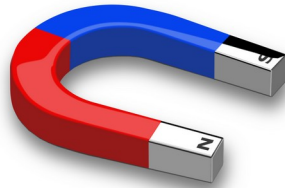
Background

- What is a bug?
 - Design defect in a software
- Do not underestimate them!
 - ~0.6% of USA GDP
 - Links to different disasters
- Guidelines for SW development in critical situations
 - MISRA C
- Some languages exclude features that easily lead to bugs
 - TrapC extension



- Syntax errors
 - Caught by the compiler
- Semantic errors
 - Uninitialized variables, dead code, typing wrong variables, **pointer/array errors**
 - Maybe warnings

- Enable as much warnings as possible
 - -Wall
- Code walkthrough
- `printf()` debugging
- `assert()` macro
- Debugging tools



- Look at warnings
 - Modify the code to solve them!
- Read again the code that is not working
 - And again
 - And again
 - And again

- Add some printf() in the portion of the code that does not work properly
- 2 main cases
 - Print variable content
 - Print something that allows to understand whether that part of code is reached

printf() debugging example



```
#include <stdio.h>

int main(int argc, char **argv){
    int sum=0, val, num=0;
    double ave;
    while (scanf("%d\n",&val) == 1) {
        sum += val;
        num++;
    }
    if(num > 0) {
        ave = sum/num;
        printf("Average is %f\n", ave);
    }
}
```

At each iteration,
sum should be the
total of entered
num values so far

At the end, *sum*
should be the total
of entered *num*
values

Ave should be the
floating point
average of input
numbers

printf() debugging example

```
#include <stdio.h>

int main(int argc, char **argv){
    int sum=0, val, num=0;
    double ave;
    while (scanf("%d\n",&val) == 1) {
        sum += val;
        num++;
        printf("val = %d sum = %d num = %d\n", val, sum, num);
    }
    printf("After while() val = %d sum = %d num = %d\n", val, sum, num);
    if(num > 0) {
        ave = sum/num;
        printf("sum = %d num = %d sum/num = %f\n", sum, num, sum/num); //we have a warning here....
        printf("Average is %f\n", ave);
    }
```

- Disadvantages
 - We need to add/remove code
 - Anyway we can use conditional compilation
 - Maybe slow
 - Output clogging
 - `stdout` is buffered
 - Use the `stderr`

- Preprocessor directive
- Widely used for conditional compilation
- Syntax:

```
#ifdef <condition>  
... <some code> ...  
#endif
```

- When the condition is false the code is simply wiped out by the preprocessor before passing it to the compiler

- Used to test a condition that **must** be always true
 - If no bugs...
- When the condition is not true
 - Print error message and terminate the program
- Good programming practice
 - Widely used in the development stage
 - Not meant to be used for catching runtime input errors
 - Can be easily disabled in the release stage
 - In our case, not very useful at exam time

assert() examples

```
#include <assert.h> // mandatory
```

```
...
```

```
assert(day_of_month <= 31);
```

```
...
```

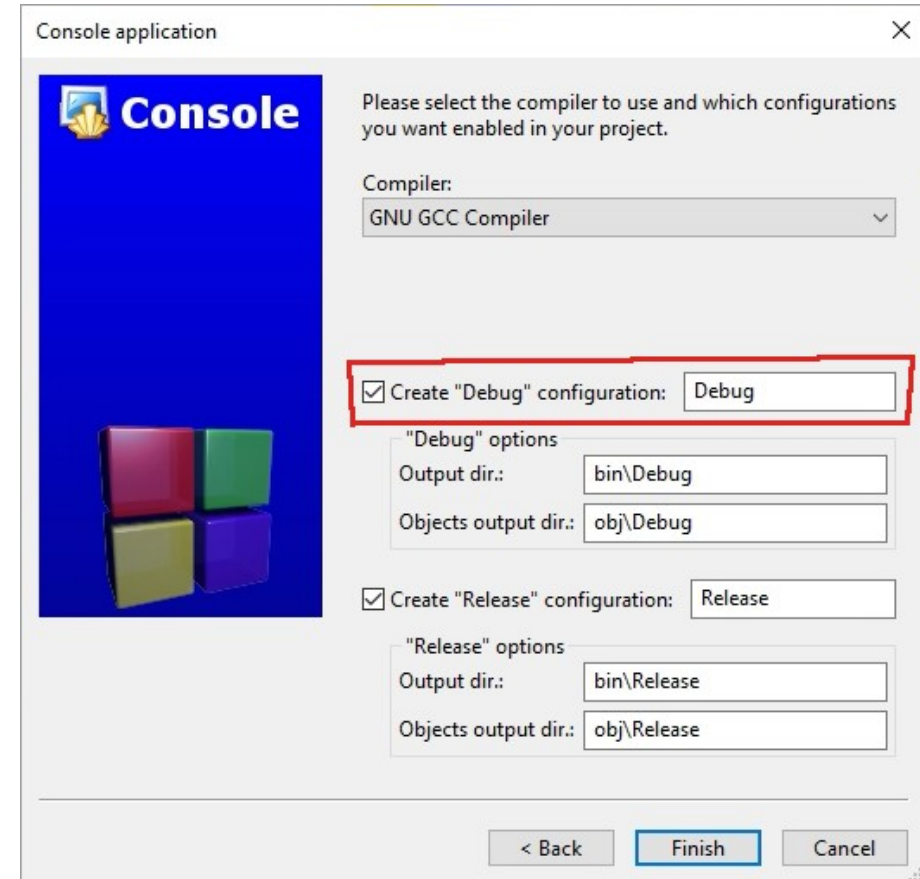
```
void resize(int *value) {  
    assert( value != NULL );  
}
```

```
...
```

- A debugger is a tool that can monitor a program execution performing:
 - Watching/inspecting variable content
 - Even change variable contents
 - Halting program on given conditions
 - Step by step execution
- The debugger **must** have access to the source code

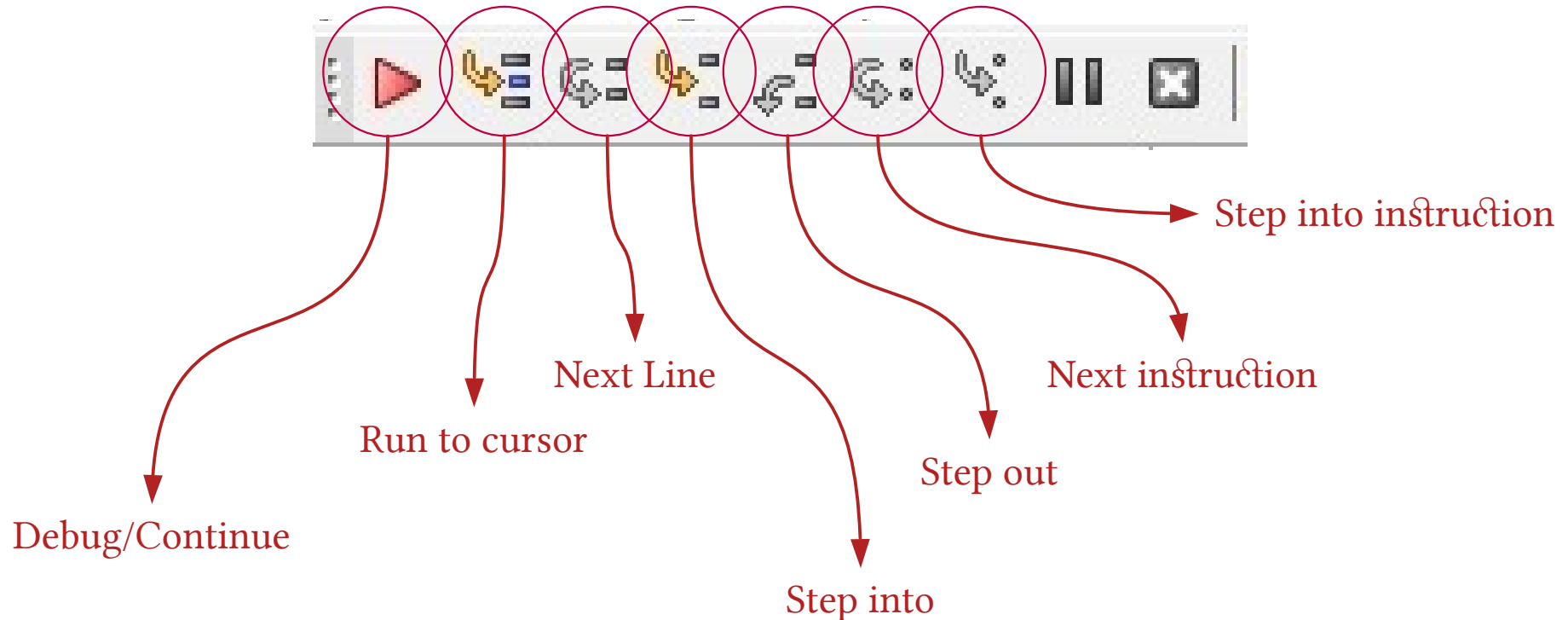
Compiling for debugging

- In order to debug a program we need to instruct the compiler to also **include references to the source code** in the executable
- Code::Blocks, by default, also create a debug executable



How to debug in Code::Blocks?

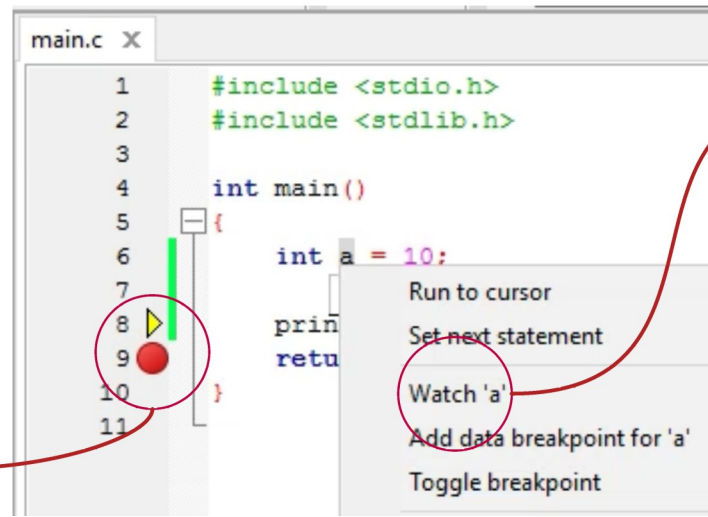
- You have to use specific commands



How to debug in Code::Blocks?

- Moreover we can also add:
 - **Breakpoints:** execution stopping places
 - **Watches:** inspection tool for showing variables code

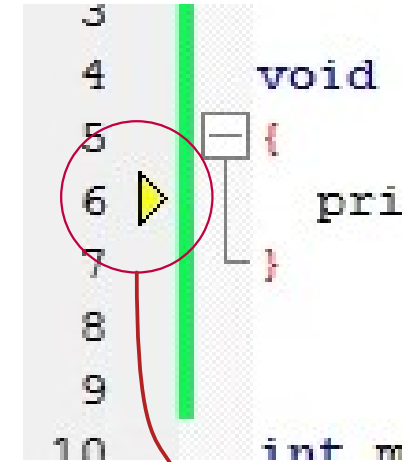
To add a breakpoint, click
on the gray part just on
the right of the line
number



Select and right-click on a
variable name to inspect
its content

How to debug in Code::Blocks?

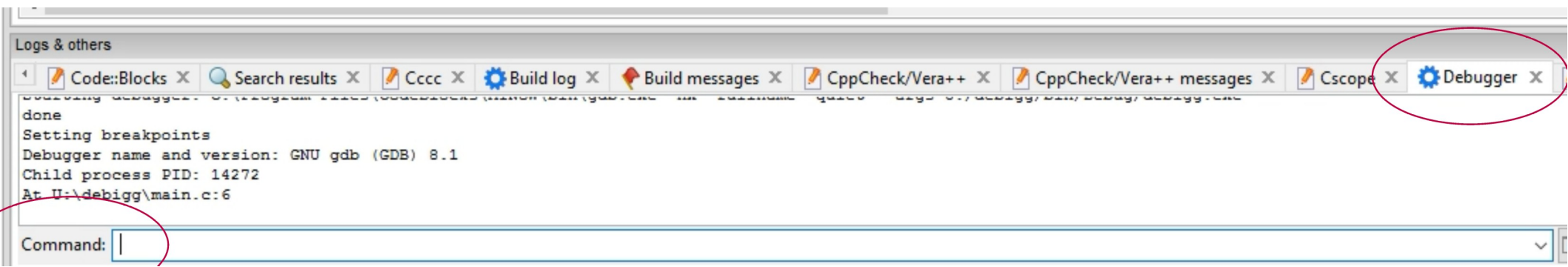
- **Debug/continue:** start or continue execution but stop at next breakpoint
 - No breakpoints → normal execution
- **Run to cursor:** start execution but stops at cursor line
- **Next line/instruction:** execution step by step
- **Step into:** step by step also entering functions



Line/instruction to be
executed next

How to debug in Code::Blocks?

- On the bottom part you can also find a Command Line Interface
- You can issue debug commands, most useful:
 - **bt**: print call hierarchy
 - **p <exp>**: i.e., “p a[i]”, print content of a variable or result of an expression
 - **w <condition>**: i.e., “w a==3”, stop the execution when the condition is not met



- Start debug where the bugs are!
- When not clear, try to understand as fast as possible where the problem is
- Binary search principle:
 - If you have a large chunk of code but you are not sure where the bug is
→ put a breakpoint in the middle
 - This allow you to maybe understand whether the bug is before or after that point

- In a “long” cycle use conditional watches
- i.e. using the “w” CLI command



UNIVERSITÀ DI PARMA

Debugging with Code::Blocks



KEEP
CALM
IT'S
QUESTION
TIME

If it ain't broken, don't fix it.

Ronald Reagan

If we can't fix it, it ain't broke.

Lt. Col. Walter Weir